Deep Residual Learning for Image Recognition

구분	Euron
날짜	@2026년 3월 7일 → 2026년 3월 9일
상태	완료
# 연도	2016
키워드	컴퓨터비전

- (이미지 인식을 위한 deep 잔차 학습)
- ResNet(Residual Network_잔차 학습)을 제안한 논문
  - CNN 깊어지면 문제 발생, Residual Learning으로 해결
  - CNN을 매우 깊게 학습할 수 있게 만든 skip connection 구조(ResNet)

Abstract

- nn deeper → train 어려워짐 ⇒ **residual learning framework** 제안
  - 각 층이 layer input을 기준으로 하는 잔차 함수(residual function)을 학습하도록 재구성 (↔ 독립적인 함수 직접 학습- 기존)
  - ⇒ 최적화 쉬움, depth ↑ accuracy
    - *ImageNet 데이터셋*에서 152층 layer 깊이에서의 residual network 평가
    - residual nets 앙상블 - 좋은 성과(3.57% 오류율)
    - *CIFAR-10 데이터셋*에서 100층 / 1000층 레이어 분석

- 표현*representation*의 깊이는 visual recognition task에서 중요
  - *COCO object detection 데이터셋*에서 28% 상대 성능 향상

1. Introduction

- Deep convolutional*합성곱* nn은 image classification 유용해 옴
  - Deep network :
    - 저/중/고차원 feature들과 classifier*분류기*를, end-to-end 다층 구조 안에서 통합
    - feature level은 레이어 쌓은 만큼=depth 풍부
      - recent evidence + ImageNet 데이터셋 성능 b 모델들은 모두 deep - 16~30층

- ⇒ Q "더 좋은 network을 만드는 것은, 단순히 더 많은 층을 쌓는 것으로 가능한가?"
  - ↔ vanishing/exploding gradients - 학습 초기 모델 수렴 어렵게 함
  - normalized initialization*정규화된 초기화*, intermediate normalization layer *중간 정규화 층*으로 상당 부분 해결
    - 수십 개 층을 가진 network도 stochastic gradient descent(SGD)*확률적 경사 하강법*과 backpropagation으로 수렴 시작 가능

- 깊은 네트워크들의 수렴으로 드러난 또 다른 문제 : **"degradation problem"**
  - **depth ↑ → accuracy 포화 → 성능 떨어짐**

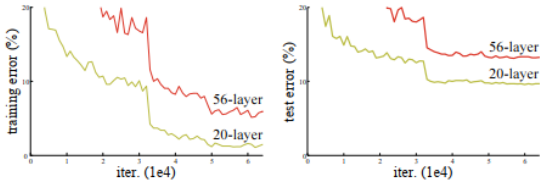


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

  - 이는 과적합 때문이 아니라, **layer 수 증가가 training error 를 증가시켜서**

- 의미 - 모든 시스템이 동일하게 최적화되기 쉬운 것이 아니다

- 얕은 네트워크 1, 깊은 네트워크1(기존에 몇개만 더 추가한)의 예시
- solution by construction 존재 ⇒ **깊은 네트워크는 최소한 얕은 네트워크와 같은 성능을 낼 수 있어야 한다**
  - 추가된 레이어는 identity mapping이고 나머지는 얕은 쪽 weight 복사하면 됨
- <u>but 실제로는 다르게 나옴</u>

- degradation problem ⇒ “deep residual learning framework” 제안! : 각 layer가 직접 원하는 함수를 학습하도록 하는 대신, 잔차 함수를 학습하도록 함
  - 예시 - **<u>원하는 함수</u>가 <u>H(x)</u>**일때
  - layer들이 학습하도록 하는 함수 :

$$F(x) = H(x) - x$$

  - 원래의 함수 :

$$H(x) = F(x) + x$$

출력 = residual함수 + 입력

F(x) → 몇 개의 layer가 만든 결과(일반 cnn구조)

```
x
 ↓
conv
 ↓
relu
 ↓
conv
 ↓
output
```

수식으로 쓰면

$$y = F(x)$$

즉

```
입력 → 여러 layer → 출력
```

- 잔차 함수 학습이 기존 직접적 함수 학습보다 쉽다고 가정
- 극단적인 경우 identity mapping이 해라면 여러 비선형층으로 학습하는 것보다 그냥 F를 0으로 만들면 됨 (easy)

- 이러한 F(x) + x 형태는 **shortcut connection**을 가진 **feedforward 신경망(nn)**으로 구현 가능!
  - shortcut connection : 하나 이상 layer 건너뛰는 연결.

    여기서는 identity mapping 수행 → stacked layers의 출력에 합쳐짐

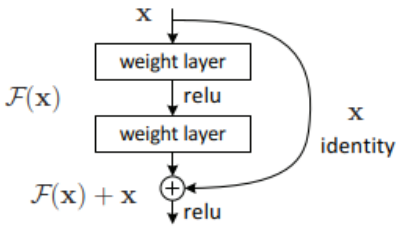


Figure 2. Residual learning: a building block.

  - 추가 파라미터, 추가 계산 복잡성 없음
  - SGD with backpropagation으로 여전히 엔투엔 학습 가능 + 기존 딥러닝 라이브러리 쉽게 구현

- 실험 후 결과

  1) deep residual nets의 최적화 쉽다
     - ↔ 일반 stack layers 방식은 deeper수록 training error 증가

  2) 깊이 증가할수록 정확도 향상

- CIFAR-10 데이터셋에서도 O, ImageNet도 152층으로 가장 깊지만 VGG보다 복잡도 낮음 + ImageNet detection + ImageNet localization + COCO detection+ COCO segmentation

- 그래서 residual learning은 일반적이며 CV 뿐 아니라 다른 문제에도 적용될 수 O....

2. Related Work

1) Residual Representations

+residual = **실제값 - 기준값** = **출력 - 입력** = **입력에서 얼마나 수정해야 하는지 = F(x)** = H(x)를 직접 배우지 말고 H(x)에서 **입력을 뺀 변화량**을 배우자

- **VLAD** : dictionary에 대해 계산된 residual vectors로 정보 인코딩하는 방식
- **Fisher Vector** : VLAD의 확률적 버전
- image retrieval*이미지 검색과* image classification에서 powerful shallow representations
- vector quantization*벡터양자화*에서는 원래 벡터 인코딩보다 잔차 벡터 인코딩이 더 효과적

▼ VLAD(추가)

- image feature aggregation 방법
- 이미지에서 feature 추출 → 벡터들 나옴

$$feature = [0.3, 0.7, 0.2]$$

- dataset에서 대표 feature인 cluster center 만듦

$$center = [0.4, 0.5, 0.6]$$

- residual = feature - center
- 이 차이벡터를 모아서 이미지 전체 표현을 만듦

2) Shortcut Connections

- 이를 사용하는 방법과 이론은 오랫동안 연구돼 왔다...

(어쨌든 shortcut을 ResNet이 처음 생각해낸건 아니다)

a. multi-layer perceptron(MLPs)를 학습시키는 초기 방법 중 하나는 **네트워크의 입력에서 출력**으로 **직접 연결되는 linear layer***선형층*을 추가하는 것이었다

b. **중간 층을 auxilary classifier*보조 분류기*에 직접 연결**하는 방법 ← gradient vanishing/exploding 완화 위해

c. **layer의 response/gradient/propagated errors**를 centering하는 방법

▼ centering(추가)

값들의 평균을 0에 가깝게 만드는 것

d. **Inception layer** - shortcut branch + a few deeper branches 구성

e. **Highway Networks -** gating 함수가 있는 shortcut connection 제안

- 이 gate는 date-dependent, parameters ↔ ResNet은 파라미터 X (차)
- gate closed=값이 0에 가까워지면 해당 layer는 non-residual fuction이 됨 ↔ ResNet은 항상 residual function 학습 : identity shortcut은 절대 closed X, 모든 정보가 항상 shortcut을 통해 전달되고 그 위해 residual 함수가 추가로 학습됨
- 매우 깊은 네트워크에서 정확도 향상 결과는 X였음

3. (Method)Deep Residual Learning

3.1 Residual Learning(아이디어)

3.2 Identity Mapping by Shortcuts(구현 방법)

3.3 Network Architectures(실제 네트워크 구조)

3.4 Implementations(학습 설정)

3.1 Residual Learning (아이디어)_왜 residual learning인가

- H(x) : stacked layers가 fit=학습해야 할 기본적인 함수, x는 첫 번째 레이어의 input
  - 만약 여러 비선형 층이 복잡한 함수를 점점 더 잘 근사할 수 있다고 가정한다면 → 이 층들이 잔차 함수를 근사할 수 있다고 가정하는 것도 같은 의미가 된다

    (표현력이 그대로다. H(x)를 표현할 수 있다면, F(x)도 표현할 수 있다)

    즉, F(x):=H(x)−x (입력과 출력의 차원이 같다고 가정할 때)
- 따라서 stacked layers 여러 층이 직접 H(x)를 학습하도록 하는 대신, 잔차 함수 F(x)를 학습하도록 설계한다
  - original function H(x) = F(x) + x

+두 방식 모두 이론적으로는 앞에서 가정한 대로 원하는 함수를 충분히 잘 근사할 수 있지만, 학습 용이성은 서로 다를 수 있음

- 잔차로의 재구성 motivate : identity mapping의 degradation 문제를 생각해 볼 때.
  - 만약 추가된 층들이 identiti mapping, 즉 입력을 그대로 출력하는 함수로 구성된다면, deeper 모델은 shallower 모델보다 training error가 크지 않아야 함 (서론 설명 부분)
  - degradation problem은 solver가 여러 개 비선형 층 이용 시 identity mapping을 찾는 데 어려움을 겪는다는 것 시사 (22)

▼ 어려운 이유(추가)

4. 이유 1 — 비선형 층이 많기 때문

deep network는 보통 이런 구조다.

```
x
↓
W1x
↓
ReLU
↓
W2x
↓
ReLU
↓
W3x
```

여기서 identity mapping을 만들려면

```
output = x
```

이 되어야 한다.

즉

```
ReLU(W3 ReLU(W2 ReLU(W1 x))) = x
```

가 되어야 한다.

이건 **weight들이 매우 특정한 값**이어야 한다.

5. 이유 2 — gradient propagation 문제

layer가 많으면

gradient가 이렇게 된다.

```
gradient = W1 × W2 × W3 × ... × Wn
```

그래서

```
vanishing gradient
exploding gradient
```

문제가 생긴다.

그러면

```Plain text
optimizer가
올바른 weight를 찾기 어려워진다
```

- 이때 residual learning으로 재구성하면, 여러 개 비선형 층의 weight를 단순히 0에 가깝게 만들기만 하면 identity mapping에 가까워질 수 있음(33)

+직관적인 비유 : 기존 네트워크는 10→10을 직접 만들어야 하는데 ResNet은output = x + F(x)이므로 F(x)=0이면 끝임

3.2 Identity Mapping by Shortcuts(구현 방법)_residual block을 어떻게 구현하는가

- 모든 stacked layers마다 residual learning 적용
  - 도식

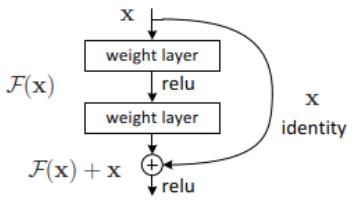


Figure 2. Residual learning: a building block.

하나의 building block

  - 형식

$$y = F(x, \{W_i\}) + x$$

- x : 그 layer의 input 벡터
- y : 그 layer의 output 벡터

a. $F(x, \{W_i\})$: 학습해야 할 residual mapping
   - fig 2에서는 두 개의 층, $F = W_2\sigma(W_1x)$
     - σ는 RELU, 표기 단순화 위해 bias 생략

b. $F + x$: shortcut connection + element-wise addition으로 수행

c. 덧셈 이후 두 번째 비선형 함수 $\sigma(y)$ 사용

- shortcut connection은 추가적인 파라미터도, 계산 복잡도도 거의 증가시키지 않음
  - plain network와 residual network를 비교할 때 동일 파라미터 수 / 동일 깊이 / 동일 폭 / 동일 계산 비용으로 공정하게 비교함

- 식 (1)이 성립하려면 x와 F의 차원이 같아야 함

  만약 그렇지 않다면, **shortcut connection에서 linear projection W_s** 사용해 차원 맞출 수 있음

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \qquad (2)$$

이 경우의 식

  - 식 (1)에서도 W_s를 사용할 수 있지만, 실험 결과 identity mapping만으로도 충분했음 + 계산 효율적
  - 따라서 W_s는 차원을 맞춰야 할 때만 사용

- 잔차 함수 F(x)의 형태는 유연함
  - 이 논문의 실험에서는 2개 or 3개 층의 F만 사용하지만 더 많은 층을 사용하는 것도 가능
  - 그러나 1개 층만 가진다면, 식 (1)은 다음과 같이 linear layer와 유사하게 됨 but 성능 향상 관찰 X *<< 2개 이상 층 쓰자*

$$y = W_1 x + x$$

- 위 표기는 단순화를 위해 fully-connected layer를 기준으로 설명했지만 동일 개념은 convolutional layer에도 적용 가능
  - 즉 $F(x, \{W_i\})$는 여러 개의 convolution layer 나타낼 수 O

▼ 설명(추가)

지금까지 수식

$$F(x) = W_2 \sigma(W_1 x)$$

이건

```
matrix multiplication
```

형태라서

```
FC layer
```

처럼 보인다.

하지만 실제 ResNet은

```
CNN
```

이다.

그래서 논문이 말하는 것이다.

> 같은 개념은 convolution layer에도 적용된다.

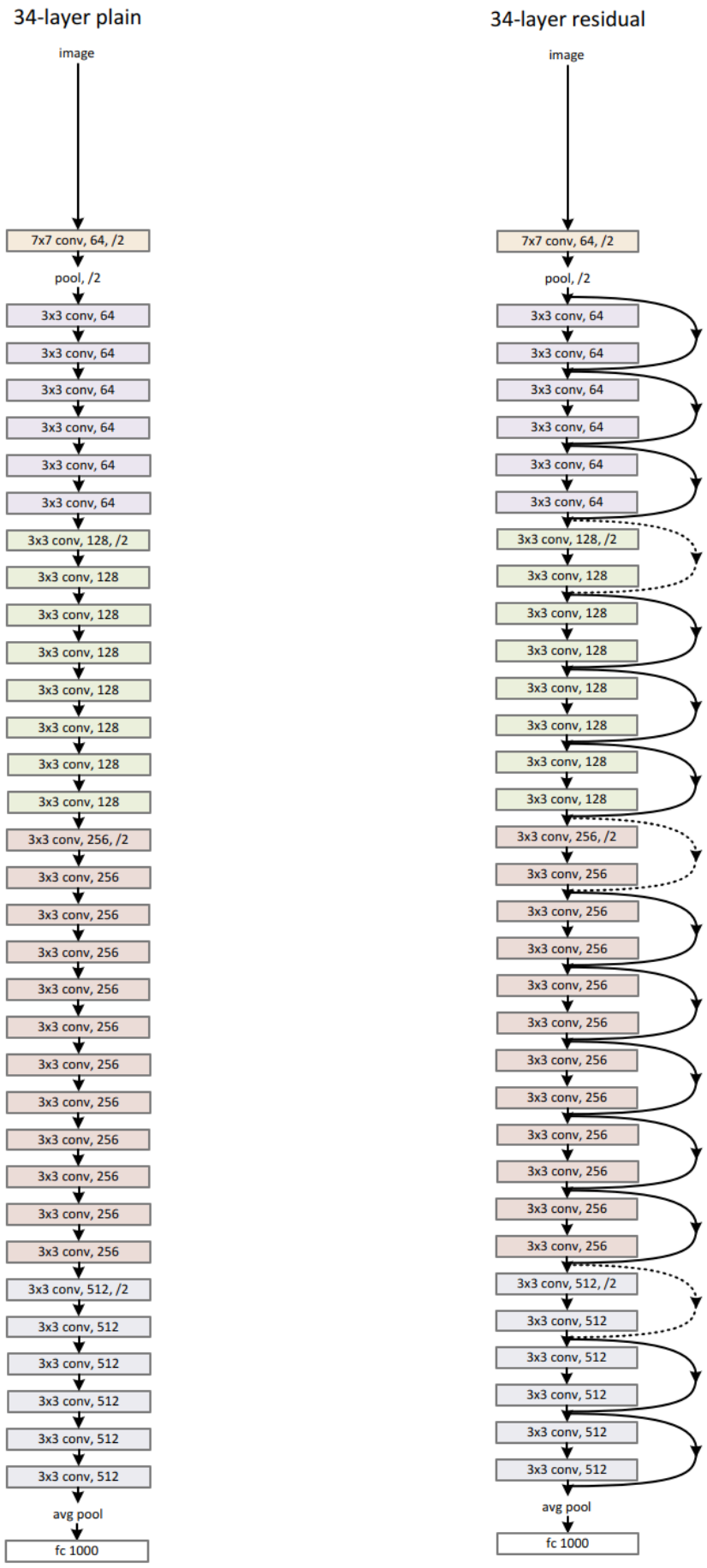
  - 그리고 element-wise addition은 두 개의 feature map을 채널 단위로 수행=각 채널마다 픽셀 단위로 수행=channel by channel

3.3 Network Architectures (실제 네트워크 구조)_실제 ResNet 구조는 어떻게 생겼는가

두개 비교. 문제는 depth가 아니라 architecture라는 것 보여줌

Plain Network : shortcut 없는 일반 CNN - baseline

Residual Network : ResNet 구조


34-layer plain
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 256, /2
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 512, /2
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
avg pool
fc 1000
34-layer residual
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
3x3 conv, 256, /2
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 512, /2
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
avg pool
fc 1000

- VGG nets 기반
  - 이것보다 이 baseline이 필터 수 적고 계산 복잡도도 낮긴(18%) 함
- convolutional layer들은 대부분 **3×3 필터**를 사용하며, 다음의 두 가지 단순한 설계 규칙을 따름
  - convolutional layer들은 대부분 **3×3 필터**를 사용하며, 다음의 두 가지 단순한 설계 규칙을 따름
  - feature map의 크기가 절반으로 줄어들면, 각 층의 연산 복잡도*time complexity*를 유지하기 위해 필터 수를 두 배로 늘림
- stride =2 인 convolution layer로 downsampling
- 네트워크 마지막 - global average pooling layer + 1000개 출력 가진 fc layer(softmax)
- 총 weighted layer 수 34개
- 옆 plain network → shortcut connection 삽입해 residul version으로 전환
  - 차원이 증가하는 경우(점선) :

    a. shortcut이 여전히 identity mapping을 수행하도록 두고, 차원이 증가하는 부분은 **0으로 padding해 채우기**

    - 추가 파라미터 도입 X

    b. 식 (2)에서 제시한 **projection shortcu**t으로 차원 맞추기 **by 1 X 1 convolution**
  - 입력과 출력의 차원이 동일한 경우(실선) : identity shortcut (식(1)) 직접 사용
  - 두 방법 모두 shortcut은 stride = 2로 수행

3.4 Implementation(학습 설정)_어떻게 학습했는가

- **Resize** = scale augmentation
  - 짧은 변의 길이 256~480 사이 랜덤
- **Random Crop** = data augmentation
  - 이미지에서 224 X 224 부분을 랜덤으로 잘라서 학습
- **Horizontal Flip**
- **Mean Subtraction**
  - 이미지에서 평균 RGB 값을 뺌
  - 데이터 정규화. 학습 안정성을 위해
- **Color Augmentation**
  - standard color augmentation으로 색상 랜덤 변경

        - brightness, contrast, saturation 등
- Batch Normalization
    - 각 conv 이후 적용
    - feature 분포 안정화, 학습 속도 증가 위해
- SGD 학습
    - optimizer : SGD
    - batch size : 256
- Learning Rate
    - learning rate = 0.1
    - error plateau 시 1/10으로 줄임
- Iteration
    - 60 × 10^4 iterations
- Weight Decay
    - weight decay = 0.0001
    - L2 regularization임. overfitting 방지
- Momentum
    - momentum = 0.9
    - SGD를 더 안정적으로 만듦, gradient 이동에 관성 추가
- Dropout 사용 X
    - BatchNorm이 regularization 역할 일부 수행했기 때문
- Testing
- Testing - 10 crop
    - 한 이미지를 여러 방식(center crop, corner crops, flip)으로 잘라서 총 10개 이미지로 분할, 예측
    - 그리고 평균을 냄
- Multi-scale testing
    - 이미지를 여러 크기로 테스트

- shorter side in {224, 256, 384, 480, 640}로 각 여러 크기에서 테스트하고 score 평균을 냄

4. Experiments

4.1 ImageNet Classification

- 평가 지표 : top-1 오류율, top-5 오류율

Plain Networks

- 18-layer와 34-layer plain network 평가
- 결과 : 더 깊은 34가 얕은 18보다 validation error 높았음
  - 이유 분석 위해 학습 과정 동안의 **training error와 validation error를 비교**
    - degradation problem 관찰됨, 전체 학습 과정 동안 34가 더 높은 training error 보임
  - 18의 해 공간*solution space*가 34의 해 공간의 부분집합임에도 불구하고 나타남
- 이 optimization difficulty최적화의 어려움이 vanishing gradient 문제 때문이 아닐 것이라고 주장
  - plain networks는 Batch Normalization을 통해 학습되었으며, BN은 순전파되는 신호가 0이 아닌 분산을 갖도록 보장
  - 역전파되는 gradient 역시 BN을 사용할 때 정상적인 norm크기가지는 것 또한 확인

  → 따라서 forward signal, backward signal 모두 사라지지 않는다

- 34도 여전히 경쟁력 있는 정확도를 달성할 수 있고, 이는 solver가 어느 정도는 제대로 작동하고 있음을 시사
  - but 깊은 plain network이 매우 느린 수렴 속도 가진다고 추측 = training error가 줄어드는 속도가 지수적으로 매우 느릴 수 있음
- 추가 원인은 향후 연구에서 더 조사될 것

Residual Networks

- 18-layer와 34-layer residual network 평가
  - 기본 architecture는 plain과 동일하지만 each pair of 3X3 filters마다 shorcut connection 추가 (구조 사진 참고)
  - 모든 shortcut에 identity mapping 사용, 차원 증가시 zero padding(option A) 사용

    → plain과 비교했을 때 추가 파라미터 X

- 결과1 : 34가 18보다 더 좋은 성능, 더 낮은 training error와 val data에서 잘 일반화
  - degradation problem이 이 설정에서 잘 해결되었음을 의미
  - 네트워크 깊이 증가함에 따라 실제로 정확도 향상을 얻을 수 있음 의미
- 34 ResNet이 plain과 비교했을 때 top-1 오류 35% 감소
  - training error가 성공적으로 감소했기 때문
  - deep network에서 residual learning이 효과적인 것 검증

- 결과2 : 18 plain과 18 ResNet은 성능 거의 비슷
  - 하지만 18 ResNet이 더 빠르게 수렴
  - 그다지 깊지 않으면 현재의 SGD optimizer도 plain에서 충분히 좋은 해를 찾을 수 있지만, ResNet은 초기 학습 단계에서 더 빠른 수렴을 제공해 최적화를 쉽게 한다

💡

1. 깊은 네트워크에서는 ResNet이 훨씬 좋다
2. 얕은 네트워크에서는 성능은 비슷하지만 학습 속도가 빠르다

Identity vs. Projection Shortcuts

- ResNet에서 shortcut을 어떻게 구현하는 것이 좋은지 비교 실험
  - Identity shortcut Vs. projection shortcut.

a. identity shortcut + dimension 증가 시 zero padding
   - 가장 단순, 추가 파라미터 X

b. identity shortcut + dimension 증가 시 1 X 1 convolution projection
    - 중간 방법
c. 모든 shortcut을 projection
    - 가장 복잡, 모든 skip connection에 1 X 1 convolution 사용

- 결과
    1. a, b, c 모두 plain보다 훨씬 좋다
        - 즉 shorcut 자체가 핵심
    2. c > b> a
        - b > a : a는 zero padding된 채널에 residual learning이 없어서
        - c > b : projection shortcut이 파라미터를 더 추가하기 때문
    3. ⭐a, b, c의 차이는 매우 적음
        - projection shortcut은 필수요소가 아니다
        - residual learning의 핵심은 skip connection 자체다

- 논문의 선택 - b
    - 성능 좋음 + 복잡도 낮음
    - + 밑 bottleneck architecure의 복잡도 증가 방지에 중요

Deeper bottleneck Architectures

ImageNet을 위한 더 깊은 네트워크.

- 감당 可 학습 시간 고려해 **building block을 bottleneck 구조로 수정**
    - 함수 F에 대해 기존 layer 2개 → 3개
        - **1×1, 3×3, 1×1 convolution**
    - +identity shorcut 말고 projection shortcut으로 대체하면 복잡도 크게 증가해서 identity shorcut으로 더 효율적이게.

- **50-layer ResNet**

    - 34-layer network의 각 2-layor block을 3-layer bottleneck block으로 교체
    - 차원 증가 시 option B(projection shortcut) 사용
    - 연산량 : 3.8 billion FLOPs
- **101-layer & 152-layer ResNets**
    - 3-layer bottleneck block을 더 많이 쌓아 구성
    - depth 크게 증가했음에도, 152-layer ResNet이 VGG-16/19보다 낮다

- 50/101/152-layer ResNet은 34-layer 모델보다 훨씬 더 높은 정확도를 보임
    - degradation problem도 관찰 X
    - 깊이 증가에 따른 이점이 모든 평가 지표에서 확인됨

Comparison with State-of-the-art Methods

- 기존 모든 모델보다 성능이 좋다

4.2 CIFAR-10 and Anaysis

ResNet이 깊은 네트워크에서 왜 잘 작동하는지 실험적으로 분석하는 부분

ImageNet → 성능 증명
CIFAR-10 → 원리 분석

ImageNet 결과만 보면 ResNet이 좋은데 하지만 왜 좋은지는 아직 증명되지 않음. 그래서 CIFAR-10에서 controlled experiment를 함 - 네크워크가 깊어질 때 plain network과 ResNet은 어떻게 다른가?

- 실험의 목적은 최고 성능을 달성하는 것이 아니라 매우 깊은 network의 동작을 분석하는 것. 따라서 의도적으로 단순한 architecture 사용

▼ 실험

- 실험 구조
    - 입력 : **32×32 이미지**
    - 첫 layer : **3×3 convolution**
    - 이후 **6n개의 3×3 convolution layer**
        - feature map size : **{32, 16, 8}**

        - 각 stage마다 **2n layers**
    - filter 수 : **{16, 32, 64}**
    - subsampling : **stride 2 convolution**
    - 마지막 구조
        - **global average pooling**
        - **10-class fully connected**
        - **softmax**
    - 전체 layer 수
        - **6n + 2**
- Residual connection
    - 각 **3×3 conv pair마다 shortcut 연결**
    - CIFAR-10에서는 **identity shortcut(option A)**만 사용
    - → **plain network와 파라미터 수 동일**

- 실험 설정
    - weight decay **0.0001**
    - momentum **0.9**
    - **batch normalization 사용**
    - **dropout 사용 X**
    - mini-batch **128**
    - learning rate
        - **0.1 시작**
        - **32k, 48k iteration에서 감소**
    - **data augmentation**
        - padding
        - random crop
        - horizontal flip
    - 깊이 실험

        - n = {3,5,7,9}
            - **20-layer**
            - **32-layer**
            - **44-layer**
            - **56-layer**

▼ 결과

- **Plain network 결과**
    - depth 증가 시
        - **training error 증가**
    - → **degradation problem 발생**
- ResNet 결과
    - depth 증가 시
        - **optimization 문제 해결**
        - **accuracy 증가**

▼ 110-layer ResNet 실험

- n = 18 → **110 layers**
- learning rate warm-up 필요
- **state-of-the-art 수준 결과 (6.43%)**
- Layer response 분석
    - ResNet은 **plain network보다 response magnitude가 작음**
    - → residual function이 **0에 가까운 함수라는 가설 지지**
    - depth 증가 시
        - **각 layer가 signal을 조금만 수정**

▼ 1202-layer ResNet 실험

- **optimization 문제 없이 학습 가능**
- training error < **0.1%**

- 그러나
  - **test error 증가**
  - → **overfitting 발생**

💡 핵심 결론

- plain network
  - depth ↑ → optimization difficulty ↑
- ResNet
  - depth ↑ → 성능 향상

→ Residual learning이 extremely deep networks의 학습을 가능하게 한다

4.3 Objection Detection on PASCAL and MS COCO

> ResNet은 classification뿐만 아니라 다른 vision task에서도 성능이 좋다

ResNet은 좋은 feature extractor=backbone다 - classification trick이 아니라 일반적인 feature learning improvement.

- detector = Faster R-CNN 고정, backbone을 VGG16→ResNet101로 바꾸면서 실험
- 다른 것(detector, training, dataset, hyperparameter) 모두 동일, 그래서 성능 향상은 better backbone 때문이라는 것이 증명됨